

Priority Inversion in Real-Time Operating Systems: Advances in Secure Scheduling for Real-Time Environments

Joseph Rissman

U09545917

April 29th, 2025

Video Presentation Link: <https://youtu.be/JG1VL1sQQoE>

Abstract

Real-time systems, crucial for applications ranging from avionics to self-driving cars, face increasing security challenges due to their integration with complex software and external networks. This paper addresses vulnerabilities arising from priority inversion, covert timing channels, and side-channel attacks, which exploit the deterministic nature of real-time systems to infer sensitive data or disrupt operations.

To mitigate these threats, various security-aware scheduling techniques are explored, focusing on balancing real-time constraints with robust security measures. Priority inversion, a key factor in many attacks, is examined, and solutions like the FT-PIP protocol and priority inheritance mechanisms are analyzed for their effectiveness in reducing information leakage. The research also delves into covert timing channels, proposing the TimeDice protocol to disrupt these hidden communication pathways through schedule randomization. Furthermore, the paper investigates defense strategies against side-channel attacks, including multimode scheduling and randomized scheduling techniques like REORDER++, which introduce unpredictability into task execution. The collective aim is to provide a comprehensive analysis of security considerations in real-time scheduling, ultimately enhancing the resilience of these critical systems.

Introduction

Real-time systems are fundamental to numerous critical applications, operating under strict timing constraints where tasks must complete by specified deadlines. To manage task execution and meet these deadlines, preemptive priority-driven scheduling algorithms, such as Rate-Monotonic Scheduling (RMS) and Deadline Driven Scheduling (DDS), are commonly employed. These schedulers assign priorities to tasks, allowing higher-priority tasks to preempt lower-priority ones to ensure timely completion. However, the interaction between tasks, particularly when they share resources, can lead to a problematic phenomenon known as priority inversion.

Priority inversion occurs when the execution of a higher-priority task is delayed by a lower-priority task. This situation typically arises due to contention for shared resources, such as data structures, tables, or hardware peripherals, which require synchronized access to maintain integrity. Synchronization mechanisms like semaphores, locks, or monitors are used to enforce mutual exclusion, ensuring that only one task accesses a shared resource at a time via critical sections. While necessary, these mechanisms can inadvertently cause blocking, a form of priority inversion where a higher-priority task must wait for a lower-priority task to release a required resource.

To illustrate this challenge, consider a scenario with three tasks: H (high priority), M (medium priority), and L (low priority), where H and L share a resource protected by semaphore S . If L locks S and enters its critical section and is then preempted by H , H will block when it subsequently attempts to lock S . Critically, if the medium-priority task M , which does not use the shared resource, becomes ready, it can preempt L while L is still holding the lock needed by H . Consequently, H remains blocked not only for the duration of L 's critical section but also for the

execution time of M and any other intermediate-priority tasks that might run. This leads to unbounded priority inversion, where the blocking duration is not merely a function of critical section lengths but can depend on the execution times of unrelated, intermediate-priority tasks (Fig. 1).

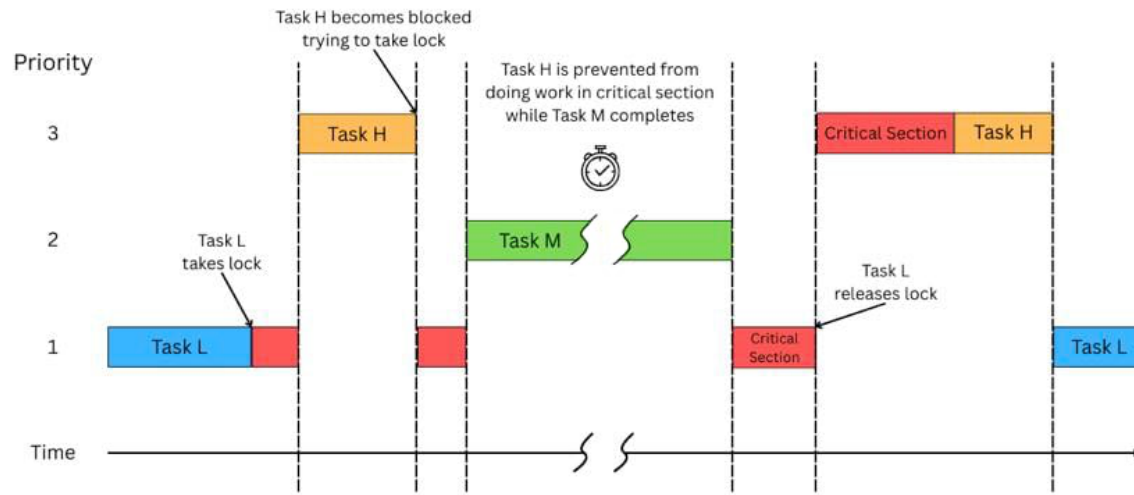


Fig 1. Unbounded Priority Inversion. Task H is blocked for the execution time of Task M, which does not share any resources with Task H.

Uncontrolled or unbounded priority inversion severely degrades the performance, predictability, and schedulability of real-time systems. It can cause high-priority tasks to miss their deadlines, even when overall system utilization is low. This issue affects both static priority systems like RMS and dynamic priority systems like DDS. While priority inversion cannot be eliminated due to the necessity of resource sharing, its duration must be minimized and bounded to ensure system reliability. To address these critical issues, this paper examines three prominent, recently developed protocols, analyzing their novel approaches to resolving priority inversion in real-time systems while also tackling the security concerns introduced by earlier solutions.

Discussion

Three recent protocols introduce novel ways to address priority inversion and related security vulnerabilities. TimeDice focuses on covert channel mitigation via randomization; FT-PIP aims to reduce multiprocessor information leakage using flush tasks and priority inheritance; and REORDER++ counters side-channel attacks through enhanced online randomization.

Before examining the specific protocols, it's important to understand a key vulnerability they address, schedule-based timing side-channel attacks. These attacks exploit the predictable, deterministic execution patterns inherent in many real-time systems. By observing timing variations, such as its own execution intervals or preemptions caused by other tasks, an attacker can infer sensitive scheduling information about victim tasks. This inferred knowledge about system internals, gained through the timing side-channel, can then be leveraged to mount more targeted and effective attacks. To combat these side-channel attacks, both TimeDice and REORDER++ leverage controlled, schedulability-aware priority inversion for randomization. TimeDice applies this between partitions in a hierarchical system to foil covert channels, whereas REORDER++ applies it between tasks within a dynamic-priority system to confuse side-channel attackers.

REORDER++ targets schedule-based side-channel attacks by enhancing schedule randomization in dynamic-priority real-time systems. It builds upon the concept of using random priority inversions to break deterministic task execution patterns, thereby obfuscating timing information that attackers might exploit. The key innovation in REORDER++ is its use of an online priority inversion test, contrasting with previous methods like REORDER that relied on offline, worst-case budget calculations. This offline analysis often proved pessimistic, especially under heavy system loads, limiting the opportunities for randomization. REORDER++ utilizes

runtime system information to more accurately assess the feasibility and available budget for priority inversions at each scheduling point. This allows for significantly more randomization opportunities, particularly in high-utilization scenarios, enhancing the system's resilience against timing inference attacks without compromising schedulability, which is verified by the online test.

Building on the need for randomization, TimeDice directly confronts the issue of timing predictability being exploited for covert channels between hierarchically scheduled real-time partitions. It recognizes that even with strict temporal isolation enforced by budget constraints, hidden information flows can emerge. The core strategy of TimeDice is to introduce randomness into the partition scheduling by dynamically and randomly inverting partition priorities at runtime. This is achieved through an online algorithm that first identifies a set of 'candidate' partitions that could run without violating the schedulability (i.e., budget guarantees) of higher-priority partitions, and then randomly selects one candidate to execute. A key challenge addressed is ensuring this randomization doesn't compromise the system's real-time guarantees; TimeDice incorporates a schedulability test within its candidacy determination to preserve the original schedulability baseline. By disrupting the temporal locality and predictability of partition execution, TimeDice aims to reduce the capacity and accuracy of covert timing channels, making it harder for adversaries to infer information based on timing variations. While effective in increasing uncertainty, this randomization can impact task responsiveness, a trade-off analyzed within the TimeDice framework.

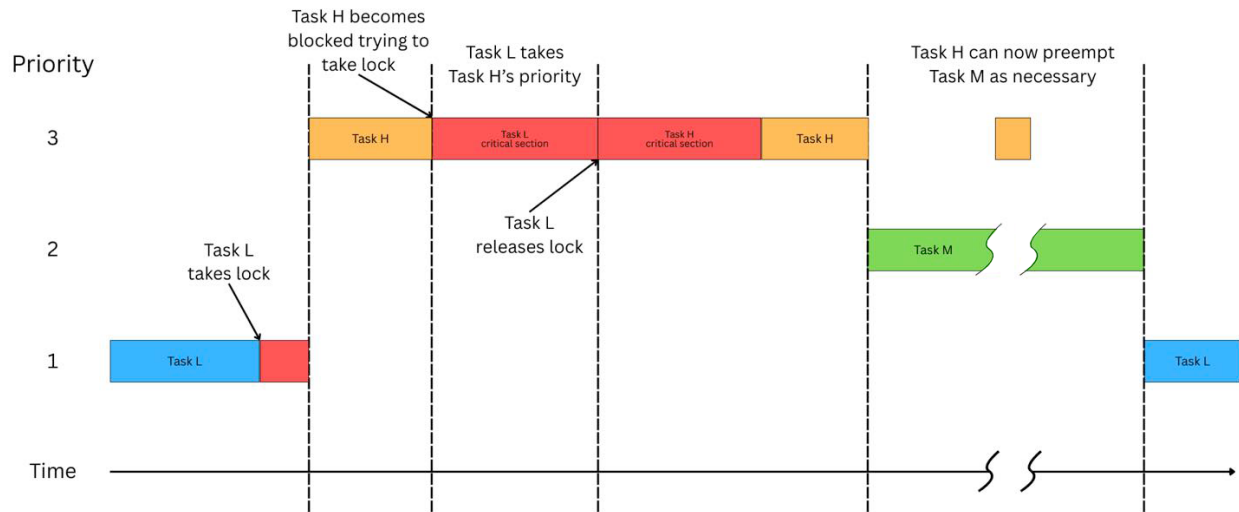


Figure 2. Priority Inheritance Protocol. Task L inherits the priority of Task H to prevent unbounded priority inversion by Task M.

Addressing security from a different perspective, the Flush Task Incorporated Priority-Inheritance Protocol (FT-PIP) focuses on preventing information leakage via shared resources in multiprocessor real-time systems. It combines the traditional Priority Inheritance Protocol (PIP) with a Flush Task (FT) mechanism. PIP is employed to manage priority inversion arising from contention over shared resources, ensuring mutual exclusion for critical sections while mitigating blocking delays (Fig. 2). The novel aspect is its integration of flush tasks, which are invoked conditionally based on a defined 'noleak' model specifying which transitions between tasks accessing a shared resource pose an information leakage risk. When a task finishes using a resource and another task begins, if a noleak relation exists between them for that resource, a flush task is executed to sanitize the resource state (e.g., clearing cache entries) preventing data remnants from being exploited in side-channel attacks. FT-PIP's contribution lies in providing a framework and associated schedulability analysis that guarantees timing constraints while satisfying security requirements for multiple shared resources on multiprocessor platforms, a scenario largely unaddressed by previous methods.

Conclusion

Enhancing the security of real-time systems against timing-based attacks necessitates innovative scheduling strategies. Protocols like TimeDice and REORDER++ leverage controlled randomization to disrupt adversarial exploitation of system predictability. TimeDice introduces schedulability-preserving randomization at the partition level, specifically targeting the mitigation of covert timing channels by adding uncertainty to partition execution sequences. Similarly, REORDER++ employs enhanced online randomization within dynamic-priority systems, dynamically assessing inversion budgets to maximize unpredictability and counter schedule-based side-channel attacks without compromising guaranteed deadlines. Both represent significant advancements in using randomization as a defense mechanism against timing inference.

Complementing randomization techniques, protocols addressing information leakage through shared resources are also crucial. The Flush Task Incorporated Priority-Inheritance Protocol (FT-PIP) tackles this by integrating resource sanitization with traditional priority inheritance mechanisms in multiprocessor environments. This dual approach manages priority inversion stemming from resource contention while actively preventing data remnants in shared resources, like caches, from being exploited. Together, these distinct approaches – randomization against timing channels and resource sanitization against leakage – illustrate the ongoing efforts to develop robust real-time systems that effectively balance stringent timing requirements with the critical need for security against sophisticated, timing-based threats.

References

1. M. -K. Yoon, J. -E. Kim, R. Bradford and Z. Shao, "TimeDice: Schedulability-Preserving Priority Inversion for Mitigating Covert Timing Channels Between Real-time Partitions," *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, Baltimore, MD, USA, 2022, pp. 453-465, doi: 10.1109/DSN53405.2022.00052.
2. H. Baek and J. Lee, "FT-PIP: Flush Task Incorporated Priority-Inheritance Protocol to Reduce Information Leakage on Multiprocessor Real-Time Systems," in *IEEE Access*, vol. 9, pp. 81882-81894, 2021, doi: 10.1109/ACCESS.2021.3086304.
3. J. Ren *et al.*, "REORDER++: Enhanced Randomized Real-Time Scheduling Strategy Against Side-Channel Attacks," in *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 6, pp. 3253-3266, Nov.-Dec. 2023, doi: 10.1109/TNSE.2023.3254653.
4. J. Ren *et al.*, "Multimode Security-Aware Real-Time Scheduling on Multiprocessors," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 3407-3418, Nov. 2024, doi: 10.1109/TCAD.2024.3445260.
5. Y. Tang, X. Jiang, N. Guan, S. Liu, X. Luo and W. Yi, "Optimizing End-to-End Latency of Sporadic Cause-Effect Chains Using Priority Inheritance," *2023 IEEE Real-Time Systems Symposium (RTSS)*, Taipei, Taiwan, 2023, pp. 411-422, doi: 10.1109/RTSS59052.2023.00042.
6. C. Sakalis, M. Sjölander and S. Kaxiras, "Seeds of SEED: Preventing Priority Inversion in Instruction Scheduling to Disrupt Speculative Interference," *2021 International*

Symposium on Secure and Private Execution Environment Design (SEED), Washington, DC, USA, 2021, pp. 101-107, doi: 10.1109/SEED51797.2021.00022.

7. J. B. Goodenough and L. Sha, "The priority ceiling protocol: A method for minimizing the blocking of high priority Ada tasks," in *Proceedings of the Second International Workshop on Real-Time Ada Issues*, Moretonhampstead, Devon, England, 1988, pp. 20–31, doi: 10.1145/58612.59371.
8. L. Sha, R. Rajkumar and J. P. Lehoczky, "Priority inheritance protocols: an approach to real-time synchronization," in *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175-1185, Sept. 1990, doi: 10.1109/12.57058.
9. S. Davari and L. Sha, 'Sources of unbounded priority inversions in real-time systems and a comparative study of possible solutions', *SIGOPS Oper. Syst. Rev.*, vol. 26, no. 2, pp. 110–120, Apr. 1992, doi: 10.1145/142111.142126.